Release 1 Implementation Roadmap and Codex Governance Plan

Document status

Purpose: Final pre-coding plan
Implementation model: One senior engineer supervising Codex AI Agent
Primary constraint: Tight budget with direct human understanding and approval
Provider status: AI model, OCR approach, visual-input strategy, cost ceilings, and raw diagnostic retention duration remain unresolved until the controlled evaluation spike
Release boundary: English-language U.S. personal auto declaration-page workflow only

---

1. Implementation strategy

1.1 Recommended implementation approach

Release 1 should be implemented through **small end-to-end vertical slices**, supported by a staged technical foundation.

A vertical slice should cross the necessary layers to create one real behavior:

- database
- backend
- authorization
- worker where needed
- frontend
- tests
- observability
- user verification

The project should not begin by building every table, every backend service, every screen, or every processing stage independently. That approach creates large amounts of technically complete but unverified infrastructure.

The recommended strategy is:

1. Establish engineering guardrails and a runnable local foundation.
2. Build the smallest authenticated customer workflow.
3. Add private file intake.
4. Add a durable job and worker lifecycle.
5. Publish a provider-neutral fixture candidate through the real candidate contract.
6. Prove review and approval behavior without live AI.

7. Run the controlled Document AI evaluation.
8. Select and approve the live provider configuration.
9. Connect the winning pipeline to the already proven review workflow.
10. complete recovery, demo, security, deployment, and polish.

1.2 Why vertical slices are better

Vertical slices reveal integration mistakes early.

For example, the slice:

> Sign in → create customer → upload PDF → see Processing

proves all of the following together:

- frontend authentication
- backend token verification
- agency authorization
- customer persistence
- private Storage authorization
- upload completion
- document creation
- attempt creation
- usage reservation
- job creation
- Processing-state presentation

Building those pieces separately would not prove that the full transaction works safely.

Vertical slices also make Codex easier to supervise because each task can be judged through visible behavior rather than internal code volume.

1.3 Provider-neutral work that can be completed first

The following can be built before selecting an AI model or OCR provider:

- repository and quality guardrails
- local development environment
- database migrations
- authentication and authorization
- customer workflow
- private file Storage
- signed upload and download
- document lifecycle
- processing attempts
- PostgreSQL job queue
- worker claims and leases

- processing-stage records
- artifact metadata
- page metadata
- pipeline-version records
- relational candidate contract
- evidence contract
- warning contract
- review state
- corrections and acknowledgements
- approval readiness
- immutable policy versions
- Current-policy selection
- audit records
- usage reservations and records
- idempotency
- concurrency
- Stored Result replay
- isolated demo sessions
- frontend review components using fixtures
- API and transaction tests

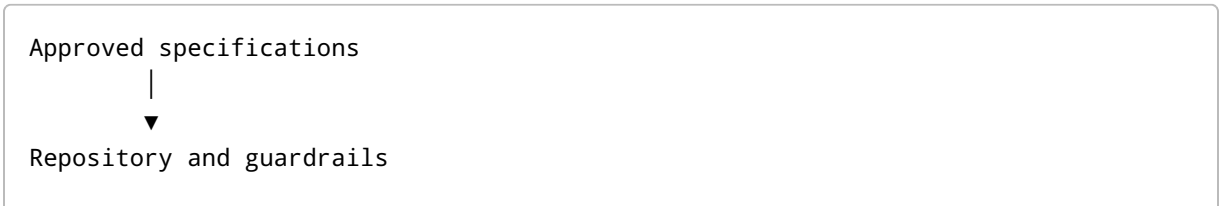
1.4 Work that must wait for evaluation

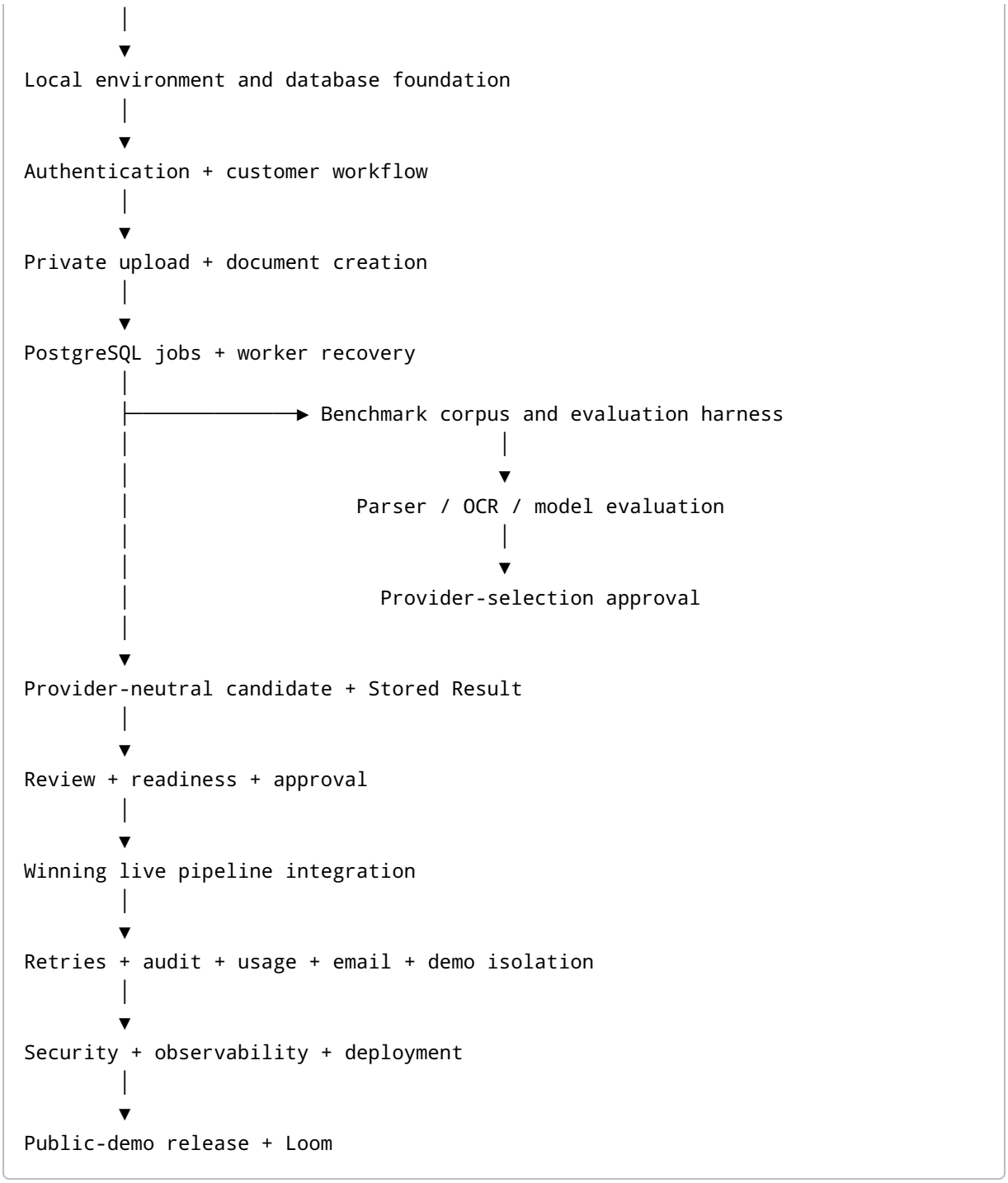
The following must not be finalized before the evaluation spike:

- final PDF parser configuration
- final OCR approach
- final OCR provider
- final AI model
- final AI-provider configuration
- text-only versus selected-page-image strategy
- provider-specific timeouts and retry thresholds
- final processing-cost ceilings
- exact raw-response retention duration
- public claims about extraction accuracy
- live-provider production integration

Provider adapters may be prototyped during evaluation, but they must not become permanent production decisions until the owner approves the evidence.

1.5 Critical path





Two streams may progress in parallel after the core database and worker contracts exist:

- provider-neutral product workflow
- controlled Document AI evaluation

They join only after provider selection.

1.6 Major dependencies

Capability	Depends on
Customer UI	Authentication, customer tables and API
Upload	Customer, Storage, upload authorization, idempotency
Worker	Attempts, jobs, leases, stage records
Candidate publication	Pipeline version, attempts, artifacts, candidate tables
Review Workspace	Candidate, evidence, warnings, source access
Approval	Review readiness, policy tables, Current selection, audit
Reprocessing	Attempts, usage, job queue, active-attempt constraint
Correction Review	Approved policy versions and review engine
Demo replay	Candidate contract, review, approval, template compatibility
Public demo	Deployment, reset, observability, Stored Results
Live Document AI	Approved provider selection and evaluation gates

1.7 Complexity intentionally avoided

The roadmap must not introduce:

- microservices
- Redis
- Kafka
- Kubernetes
- GraphQL
- event sourcing
- CQRS
- generalized workflow engines
- runtime provider routing
- dynamic extraction schemas
- multi-organization tenancy
- configurable roles
- arbitrary email templates
- customer-facing public APIs
- document chat
- Voice AI
- telephony
- billing
- subscription management

1.8 Relative effort scale

The roadmap uses relative sizes rather than calendar promises:

- **XS:** narrowly bounded change
- **S:** small integrated behavior
- **M:** meaningful vertical slice
- **L:** several coordinated work packages
- **XL:** evaluation-dependent or broad release milestone

---

2. Final approval checklist before coding

2.1 Decisions already approved

- Release 1 product scope and exclusions
- One-agency Release 1 context
- Personal auto declaration pages only
- Supported extraction fields
- Human review before approval
- Immutable approved policy versions
- Separate Current-policy selection
- FastAPI as business authority
- Next.js frontend
- PostgreSQL primary database
- Supabase Auth, PostgreSQL, and private Storage
- Vercel frontend hosting
- Render backend and worker direction
- PostgreSQL-backed durable jobs
- One orchestration job per processing attempt
- Durable processing-stage records
- Adaptive polling
- Optimistic concurrency
- Integer row versions exposed through ETag-style tokens
- Backend idempotency
- Private PDFs and short-lived signed access
- Relational candidate model plus immutable JSON snapshot
- Separate candidate, review, and approved records
- Configurable raw-artifact expiry
- Stored Result replay through the real workflow
- Disposable isolated demo sessions
- Evaluation results outside the production database
- One production provider per capability after evaluation

2.2 Decisions approved through this roadmap

Approval of this document should approve:

- one Git repository containing the coordinated frontend, backend, and worker application
- separately deployable frontend, web backend, and worker runtimes
- vertical-slice implementation order
- provider-neutral Stored Result slice before final provider integration
- granular Codex work packages
- mandatory owner gates
- migration-first persistence discipline
- contract tests beginning with the first relevant milestone
- generated fixture candidates for product development
- public Stored Results as the default demonstration path
- no public Loom until the release-readiness gate passes

2.3 Decisions intentionally deferred to evaluation

- final parser configuration
- final OCR implementation
- final OCR provider
- final AI model
- final model configuration
- selected-page-image use
- exact provider timeout values
- exact internal provider retry behavior
- processing-cost ceilings
- raw diagnostic retention duration
- quality claims shown publicly

2.4 Decisions that must never be delegated to Codex

Codex must not choose:

- product scope
- supported fields
- lifecycle states
- approval rules
- warning severity semantics
- Current-policy behavior
- architecture boundaries
- candidate-versus-approved separation
- database ownership relationships
- API resource semantics
- retry limits
- cost ceilings
- provider winner

- security claims
- privacy claims
- public quality claims
- demo restrictions
- whether a failed benchmark is accepted
- whether a migration is destructive
- whether a large architectural dependency is introduced
- whether Release 1 is ready for public launch

2.5 First-task readiness checklist

Before the first Codex task:

- All seven planning documents are stored in a stable project location.
- This roadmap is approved.
- The repository strategy is approved.
- The project's supported runtime versions are selected.
- Local secret-handling rules are written.
- Synthetic data only is available.
- The owner understands the web, worker, database, Storage, and provider boundaries.
- No production provider key is required for the first task.

---

3. Recommended phase order

The requested phase order is adjusted slightly to support earlier integrated testing.

The principal adjustment is:

> A minimal provider-neutral candidate and Stored Result path begins before provider selection. Full public-demo isolation and polished replay behavior remain later.

Phase 0 — Final decisions and project preparation

Confirm the roadmap, source documents, decision log, release boundary, and owner responsibilities.

Phase 1 — Repository and engineering guardrails

Create a runnable, reviewable repository with quality checks and strict contribution rules.

Phase 2 — Local development and environment foundation

Establish local frontend, backend, worker, PostgreSQL, Auth, and private Storage development behavior.

Phase 3 — Database migrations and core domain foundation

Introduce the approved relational schema in dependency order.

Phase 4 — Authentication and authorization

Connect Supabase identity to FastAPI agency and demo authorization.

Phase 5 — Application shell, customer workflow, and basic UI system

Deliver sign-in, Overview shell, Customer List, customer creation, and customer details.

Phase 6 — Private Storage and signed upload flow

Deliver direct private upload, backend completion, validation, and document creation.

Phase 7 — PostgreSQL job queue and worker foundation

Deliver durable jobs, claims, leases, heartbeats, stage commits, and recovery.

Phase 8 — Provider-neutral candidate fixture, benchmark corpus, and evaluation harness

Create the canonical fixture path, benchmark manifests, ground-truth format, evaluation runner, and first Stored Result candidate.

Phase 9 — Parser, OCR, model, and input-strategy evaluation spike

Compare the approved shortlist without productionizing a winner.

Phase 10 — Provider-selection gate

Review benchmark evidence and approve one pipeline configuration.

Phase 11 — Approved live Document AI pipeline

Implement the winning parser, OCR, model, context, evidence, validation, and artifact configuration.

Phase 12 — Candidate, evidence, validation, and warning publication

Connect live and Stored Result outputs to the immutable relational candidate contract.

Phase 13 — Review Queue and Human Review Workspace

Deliver operational review, source evidence, field corrections, warning resolution, and readiness.

Phase 14 — Approval and immutable policy versions

Deliver transactionally safe approval, Current/Historical behavior, and policy history.

Phase 15 — Technical Retry, Reprocessing, Correction Review, and concurrency

Deliver recovery and advanced lifecycle behavior.

Phase 16 — Audit, usage controls, and acknowledgement email

Deliver business history, cost controls, and post-approval communication.

Phase 17 — Stored Results and isolated demo sessions

Complete public-demo seeding, session isolation, replay, reset, and safe simulated actions.

Phase 18 — Security, observability, cleanup, and failure recovery

Harden access, retention, diagnostics, health, cleanup, and recovery.

Phase 19 — End-to-end testing and deployment

Deploy a production-like public-demo environment and pass complete contract and browser tests.

Phase 20 — Loom-ready visual polish and public-demo release

Complete responsive, accessibility, sample-data, visual, and demonstration polish.

---

4. Milestone specifications

Milestone 0 — Final decisions and project preparation

Relative size: XS

Objective

Convert the seven planning documents into a frozen implementation baseline.

User-visible outcome

None. This is a governance milestone.

Included scope

- Source-document catalog
- Decision log
- unresolved provider-decision list
- naming conventions
- environment names
- owner approval checklist
- Codex governance rules
- release boundary

Excluded scope

- Repository code
- provider selection
- database creation
- UI implementation

Dependencies

All seven approved planning documents.

Database work

None.

Backend work

None.

Worker work

None.

Frontend work

None.

Test requirements

No automated tests.

Verify that every source document has a stable title and revision identity.

Security checks

Confirm that no real insurance document or secret is stored in planning materials.

Expected artifacts

- Baseline decision log
- source-document index
- unresolved-decision list
- implementation-notes template

Definition of Done

- No unresolved product or architecture ambiguity blocks repository setup.
- Provider decisions are clearly marked deferred.
- Codex is prohibited from resolving them.
- Owner approves the baseline.

Human approval checkpoint

The owner should be able to explain:

- the product boundary
- the main workflow
- why candidate data is not approved data
- why provider selection is delayed

Rollback or recovery

Update the roadmap before coding. Do not compensate for ambiguity in implementation.

Interview explanation

Explain why the project began with explicit decision control rather than AI-generated code.

Milestone 1 — Repository and engineering guardrails

Relative size: S

Objective

Create the smallest runnable repository with strict quality and AI-agent boundaries.

User-visible outcome

A minimal frontend and backend can run locally and report healthy status.

Included scope

- One repository
- separately runnable frontend, backend, and worker entry points
- basic dependency management
- formatting
- linting
- type checking
- test commands
- environment-variable validation
- no-secret rules
- contribution and Codex rules
- minimal health resource
- continuous-integration checks

Excluded scope

- authentication
- database business schema
- real UI
- document processing
- provider packages

Dependencies

Milestone 0.

Database work

None beyond optional connectivity placeholder.

Backend work

- Minimal FastAPI application

- health response
- configuration validation
- request-ID foundation

Worker work

- Minimal worker process that starts and reports configuration readiness
- no queue claim yet

Frontend work

- Minimal Next.js application
- health or development landing view
- backend-connectivity indicator

Test requirements

Blocking:

- frontend checks pass
- backend checks pass
- worker process starts
- configuration failures are clear
- no secret is committed

Security checks

- committed example environment file contains placeholders only
- application refuses missing required local configuration
- dependency additions are reviewed

Expected artifacts

- runnable repository
- quality commands
- CI workflow
- engineering README
- Codex operating rules

Definition of Done

A clean checkout can run the basic application through documented local steps.

Human approval checkpoint

Inspect:

- dependency list
- service boundaries

- quality commands
- CI result
- environment handling

Rollback or recovery

Remove unnecessary dependencies or generated structure before building on it.

Interview explanation

Explain why a single repository was chosen while retaining independently deployable runtimes.

Milestone 2 — Local development and environment foundation

Relative size: M

Objective

Provide a reproducible local environment for frontend, backend, worker, PostgreSQL, Auth, and private file behavior.

User-visible outcome

A developer can start the complete local system and see service health.

Included scope

- local PostgreSQL or Supabase-compatible environment
- local Auth behavior
- private development Storage
- database connection management
- migration runner
- local synthetic-data rules
- test database
- worker database access
- frontend/backend origin configuration
- local email simulation

Excluded scope

- real provider calls
- public deployment
- production secrets
- business tables beyond migration foundation

Dependencies

Milestone 1.

Database work

- migration framework
- migration metadata
- development and test database separation

Backend work

- database health
- Auth verification configuration
- Storage service boundary
- environment classification

Worker work

- database connectivity
- Storage connectivity
- safe shutdown foundation

Frontend work

- environment-aware backend connection
- local Auth configuration

Test requirements

Blocking:

- migration can apply to empty database
- test database can reset
- private object cannot be read publicly
- frontend can reach backend
- worker can reach database
- missing secrets fail safely

Security checks

- local secrets excluded from version control
- service credentials never appear in browser configuration
- development and test databases cannot point to public production accidentally

Expected artifacts

- local setup guide
- environment matrix

- migration execution command
- reset procedure
- secret inventory

Definition of Done

One documented local startup path reliably starts all required Release 1 services without live AI, OCR, or email delivery.

Human approval checkpoint

The owner must understand:

- which credentials belong in the browser
- which remain server-only
- how local, test, demo, and future production differ

Rollback or recovery

Keep a minimal non-container fallback if local orchestration becomes unnecessarily complicated.

Interview explanation

Explain how environment separation prevents demo or test data from reaching future customer production.

Milestone 3 — Database migrations and core domain foundation

Relative size: L

Objective

Implement the approved schema in dependency order without premature feature behavior.

User-visible outcome

None initially, but later slices have stable persistence contracts.

Included scope

- identity and agency
- customers
- files and uploads
- pipeline versions
- documents and attempts
- jobs and stage records

- artifacts and pages
- candidates
- evidence, validation, warnings
- reviews
- policy versions
- audit
- usage
- email
- demo
- idempotency
- required indexes and constraints in controlled stages

Excluded scope

- complete CRUD APIs
- live processing
- UI
- production seed data
- destructive migration tooling

Dependencies

Milestone 2.

Database work

All foundational migrations described in Section 7.

Backend work

- persistence access boundaries
- transaction helper foundation
- identifier and timestamp conventions
- no generic repository abstraction that hides aggregate ownership

Worker work

Ability to read and write only worker-owned tables after later tasks.

Frontend work

None.

Test requirements

Blocking:

- foreign keys

- unique constraints
- check constraints
- partial unique active-attempt constraint
- one Current-policy relation
- valid demo ownership
- immutable-table update restrictions through application contracts
- migration from empty database

Security checks

- browser roles cannot directly access business tables
- sensitive tables have no public access path

Expected artifacts

- migration history
- schema catalog verification
- constraint test report
- migration-order notes

Definition of Done

The implemented schema matches Document 6’s consolidated table catalog and invariants.

Human approval checkpoint

Inspect:

- schema diagram
- candidate/review/policy separation
- Current-policy selection
- attempt/job separation
- demo-session ownership
- JSONB usage

Rollback or recovery

Before feature data exists, revise flawed migrations rather than layering compensating migrations. After shared environments exist, use forward-only fixes.

Interview explanation

Explain why core business truth is relational while provider output and geometry use restricted JSON or artifacts.

Milestone 4 — Authentication and authorization

Relative size: M

Objective

Make FastAPI the authority for staff and demo access.

User-visible outcome

A valid user can sign in and reach a protected application shell. Invalid or unauthorized users receive correct outcomes.

Included scope

- Supabase token verification
- application user mapping
- agency membership
- demo-session identity
- protected routes
- sign-out behavior
- expired-session handling
- object-level authorization foundation

Excluded scope

- invitations
- password-reset UI
- custom roles
- multi-agency switching

Dependencies

Milestones 2 and 3.

Database work

- identity records
- membership records
- seed one agency context
- test actors

Backend work

- token verification
- actor resolution
- agency resolution
- demo resolution

- 401/403/404 rules
- authorization test helpers

Worker work

- system and worker actor identities for audit
- no end-user token use

Frontend work

- sign-in
- protected navigation
- session-expired return behavior
- account utility
- Demo indicator when applicable

Test requirements

Blocking:

- invalid token
- expired token
- valid token without membership
- staff object access
- cross-demo-session denial
- service credential absent from browser

Security checks

- token claims are not treated as object authorization
- direct table access remains blocked
- actor identity cannot be supplied by request body

Expected artifacts

- Auth contract tests
- authorization decision matrix
- sign-in walkthrough

Definition of Done

All protected endpoints resolve one authoritative actor context before business logic.

Human approval checkpoint

Inspect a successful staff request, forbidden request, expired request, and demo cross-session request.

Rollback or recovery

Disable protected feature access if actor resolution is uncertain. Do not allow temporary permissive fallbacks.

Interview explanation

Explain authentication versus authorization and why Supabase identity alone is insufficient.

Milestone 5 — Application shell, customer workflow, and basic UI system

Relative size: M

Objective

Deliver the first complete business vertical slice.

User-visible outcome

A signed-in user can:

- open Overview
- view Customers
- create a customer
- open Customer Details
- edit contact information
- see duplicate warnings

Included scope

- application shell
- navigation
- Overview placeholder operational cards
- Customer List
- search
- create form
- duplicate check
- Customer Details Summary
- contact edit
- responsive and accessible form behavior
- idempotent customer creation
- ETag customer update

Excluded scope

- documents
- current policy
- advanced activity
- extraction
- demo templates beyond basic session context

Dependencies

Milestone 4.

Database work

Customer records, search values, audit events, idempotency.

Backend work

Customer endpoints and Overview foundation.

Worker work

None.

Frontend work

Complete customer interface.

Test requirements

Blocking:

- customer creation
- repeated creation
- duplicate warning
- stale customer update
- search
- authorization
- empty/loading/error states
- keyboard form use

Security checks

- no arbitrary agency ID in request
- normalized search does not leak cross-session records
- validation errors avoid unnecessary PII

Expected artifacts

- working vertical slice
- API contract examples
- browser test
- accessibility notes

Definition of Done

The customer workflow works through the real database and API, without mock business data.

Human approval checkpoint

Create and edit a customer manually. Trigger a stale update with two browser sessions.

Rollback or recovery

Feature can remain hidden behind authenticated development access if UX is incomplete, but data contracts must not be bypassed.

Interview explanation

Explain optimistic concurrency and why duplicate detection warns instead of automatically merging customers.

Milestone 6 — Private Storage and signed upload flow

Relative size: L

Objective

Safely accept one private PDF and create a durable Processing workflow.

User-visible outcome

A staff user can select a customer, upload a PDF, and see Processing.

Included scope

- upload authorization
- server-generated object path
- direct private Storage transfer
- completion confirmation
- content-type inspection
- byte-size check
- page-count check

- locked or malformed PDF rejection
- content hash
- duplicate warning
- document creation
- initial attempt
- usage reservation
- job creation
- audit
- source viewing

Excluded scope

- OCR
- AI
- candidate publication
- public demo local uploads

Dependencies

Milestones 3–5.

Database work

File objects, upload authorization, documents, attempts, usage reservation, jobs, audit.

Backend work

Signed upload and source-access contracts.

Worker work

No processing beyond optional safe inspection verification.

Frontend work

Upload page, progress states, duplicate warning, Processing Detail shell, PDF viewer access.

Test requirements

Blocking:

- arbitrary path rejection
- expired authorization
- repeated completion
- malformed PDF
- locked PDF
- file too large
- too many pages

- duplicate detection
- transactional rollback
- private download

Security checks

- no public URL
- signed URL expiry
- no browser service key
- filename treated as label
- active PDF content not executed

Expected artifacts

- upload sequence diagram verified
- Storage policy test
- transactional completion test
- PDF viewer proof

Definition of Done

A valid PDF creates exactly one document, one initial attempt, one usage reservation, and one durable job.

Human approval checkpoint

Inspect database records and Storage object after:

- success
- invalid file
- duplicate file
- repeated completion

Rollback or recovery

Expired or rejected uploads are cleaned without deleting accepted business files.

Interview explanation

Explain why upload transfer is direct to Storage but completion remains a FastAPI transaction.

Milestone 7 — PostgreSQL job queue and worker foundation

Relative size: L

Objective

Prove durable asynchronous work and safe recovery.

User-visible outcome

Processing Detail shows stage progress even when the browser closes.

Included scope

- job claim
- lease
- heartbeat
- job execution records
- stage execution records
- worker identity
- safe reclaim
- stale-attempt detection
- worker shutdown
- duplicate execution protection
- status polling

Excluded scope

- real OCR
- real AI
- final pipeline artifacts
- complex job prioritization

Dependencies

Milestone 6.

Database work

Jobs, executions, stages, indexes, lease fields.

Backend work

Status and processing-detail resources.

Worker work

Full claim and recovery engine using a synthetic test processor.

Frontend work

Processing stage timeline and adaptive polling.

Test requirements

Blocking:

- two-worker claim race
- expired lease reclaim
- duplicate stage completion
- worker termination
- handler incompatibility
- poisoned job
- browser closure
- polling completion deduplication

Security checks

- job payloads contain IDs, not secrets or document content
- worker cannot process a document outside its expected agency context
- diagnostic output excludes PII

Expected artifacts

- worker-recovery test report
- stage lifecycle walkthrough
- job compatibility notes

Definition of Done

A synthetic processing job survives worker interruption and completes each durable stage at most once.

Human approval checkpoint

Terminate the worker during processing, restart it, and inspect:

- lease expiry
- execution history
- stage reuse
- final attempt state

Rollback or recovery

Disable claim processing while preserving queued jobs. Never delete jobs to hide failures.

Interview explanation

Explain attempt versus job, and why stage commits make one long orchestration job recoverable.

Milestone 8 — Provider-neutral candidate fixture, benchmark corpus, and evaluation harness

Relative size: L

Objective

Prove the candidate contract and create the tooling needed for provider selection.

User-visible outcome

A fixture or Stored Result can create a real Needs Review workflow without live providers.

Included scope

- synthetic benchmark documents
- benchmark manifest
- ground-truth format
- pipeline-version fixture
- canonical candidate fixture
- candidate relational publication
- evidence fixture
- validation and warning fixture
- first Stored Result template
- evaluation runner
- result reporting format
- private benchmark artifact handling

Excluded scope

- final provider code
- production prompt
- public benchmark claims
- automated learning

Dependencies

Milestones 3 and 7.

Database work

Candidate, evidence, validation, warning, pipeline-version, and template records.

Backend work

Candidate and review-reading contracts sufficient for fixture inspection.

Worker work

Stored Result or fixture publication path.

Frontend work

Minimal candidate-inspection view may be used, but complete Review Workspace waits.

Test requirements

Blocking:

- fixture contract validation
- candidate immutability
- evidence ownership
- warning-signal linkage
- Stored versus live classification
- template compatibility

Security checks

- synthetic documents only in public fixtures
- private evaluation files isolated
- no benchmark source embedded in logs

Expected artifacts

- benchmark manifest
- ground-truth examples
- candidate fixture
- first Stored Result
- evaluation report template

Definition of Done

A known synthetic PDF can create a real candidate, evidence, warnings, and Needs Review state without calling OCR or AI.

Human approval checkpoint

Inspect every relational record created from the fixture and compare it with the source document and canonical contract.

Rollback or recovery

Regenerate fixtures if the candidate contract changes. Do not patch published templates manually.

Interview explanation

Explain why product workflow development does not need to wait for provider selection.

Milestone 9 — Parser, OCR, model, and input-strategy evaluation spike

Relative size: XL

Objective

Generate evidence for provider and pipeline selection.

User-visible outcome

No production feature. The owner receives a decision-quality evaluation report.

Included scope

- parser comparison
- OCR comparison
- model comparison
- text-only comparison
- selected-image comparison
- evidence verification
- repeatability
- cost
- latency
- hidden holdout
- failure corpus
- privacy review
- provisional retention observations

Excluded scope

- production adapters
- automatic routing
- public claims
- UI polish
- final provider selection by Codex

Dependencies

Milestone 8.

Database work

No production benchmark tables.

Pipeline-version drafts may be recorded.

Backend work

Small reusable evaluation boundaries only.

Worker work

Evaluation runners may reuse worker-compatible provider boundaries but must remain clearly experimental.

Frontend work

None required.

Test requirements

Blocking:

- benchmark manifest validation
- ground-truth validation
- fixed configuration identity
- repeatability execution
- raw result preservation
- no hidden holdout leakage

Security checks

- only permitted documents
- provider data-handling reviewed
- benchmark artifacts private
- no provider keys in reports

Expected artifacts

- parser comparison
- OCR comparison
- model comparison
- input-strategy comparison
- cost and latency report
- error inventory
- holdout result
- proposed provider decision

Definition of Done

At least one configuration passes or a clear Revise/Reject result explains why none may proceed.

Human approval checkpoint

The owner personally inspects every:

- silent critical error
- policy-number error
- VIN error
- wrong vehicle association
- unsupported acceptance
- fabricated evidence result
- repeatability variation

Rollback or recovery

Discard experimental integration code that is not part of the selected configuration.

Interview explanation

Explain why provider choice was based on a controlled benchmark rather than price, popularity, or model marketing.

Milestone 10 — Provider-selection gate

Relative size: XS

Objective

Convert evaluation evidence into one approved pipeline configuration.

User-visible outcome

None directly.

Included scope

- Go, Revise, or Reject decision
- approved pipeline-version record
- selected provider per capability
- selected input representation
- approved cost ceilings
- raw-retention duration
- expected latency thresholds

- fallback behavior
- decision-log update

Excluded scope

- implementation changes
- multiple production providers
- automatic fallback

Dependencies

Milestone 9.

Database work

Approve one immutable pipeline-version record.

Backend, worker, frontend work

None.

Test requirements

Evaluation gates must be complete.

Security checks

Review provider data handling and selected artifact retention.

Expected artifacts

- signed decision record
- winning configuration
- rejected alternatives
- known limitations
- productionization boundaries

Definition of Done

One provider configuration is approved or the spike returns to revision.

Human approval checkpoint

Owner explicitly approves:

- provider
- configuration
- cost ceiling

- retention
- quality results
- known limitations

Rollback or recovery

No provider-dependent production milestone starts without approval.

Interview explanation

Explain the selection matrix and accepted trade-offs.

Milestone 11 — Approved live Document AI pipeline

Relative size: XL

Objective

Productionize only the winning pipeline configuration.

User-visible outcome

A supported PDF can progress through real processing stages.

Included scope

- parser adapter
- text-quality evaluation
- OCR selection
- approved OCR path
- context preparation
- approved AI call
- structural validation
- normalization
- business validation
- evidence preparation
- warnings
- usage capture
- artifact retention
- terminal failures

Excluded scope

- alternative production provider
- runtime routing
- provider experimentation in normal processing

- automatic approval

Dependencies

Milestone 10 and worker foundation.

Database work

Use existing attempts, stages, artifacts, pages, usage, and pipeline-version records.

Backend work

Processing and failure-detail responses.

Worker work

Complete live orchestration.

Frontend work

Real stage descriptions and failure recovery actions.

Test requirements

Blocking:

- fixture regression
- approved benchmark
- provider timeout
- malformed output
- structural repair limit
- OCR failure
- unsupported document
- multiple policies
- artifact expiry
- usage finalization
- worker restart

Security checks

- minimum required provider content
- raw responses restricted
- PII-free logs
- provider secrets server-only
- retention applied

Expected artifacts

- live pipeline
- pipeline-version trace
- operational runbook
- benchmark regression integration

Definition of Done

The selected pipeline meets approved evaluation gates through the worker and real attempt lifecycle.

Human approval checkpoint

Run representative clean, scanned, unsupported, and failure documents.

Rollback or recovery

Retain the last approved pipeline version and disable the new one if regression gates fail.

Interview explanation

Explain each processing stage and how failures preserve work and usage.

Milestone 12 — Candidate, evidence, validation, and warning publication

Relative size: L

Objective

Connect live pipeline output to the same immutable candidate contract used by Stored Results.

User-visible outcome

A successfully processed document becomes Needs Review with evidence and warnings.

Included scope

- candidate publication gate
- candidate policy fields
- insureds
- vehicles
- drivers
- coverages
- alternatives

- evidence
- validation signals
- generated warnings
- review creation
- document state
- audit event

Excluded scope

- reviewer editing
- approval
- candidate mutation

Dependencies

Milestones 8 and 11.

Database work

Candidate insertion transaction and indexes.

Backend work

Candidate read resources.

Worker work

Transactional candidate publication.

Frontend work

Needs Review summary and read-only candidate display.

Test requirements

Blocking:

- immutable candidate
- evidence-page ownership
- target ownership
- no unsupported fields
- warning generation
- publication rollback
- Stored/live parity
- no automatic approved policy creation

Security checks

- normal staff cannot access raw provider artifacts
- evidence excerpts are limited to necessary content

Expected artifacts

- candidate publication contract test
- Stored/live comparison
- warning inventory

Definition of Done

Both Stored Result and live attempts publish through one candidate and review contract.

Human approval checkpoint

Compare one live and one Stored Result candidate through the database and API.

Rollback or recovery

Failed publication leaves the attempt Failed rather than partially reviewable.

Interview explanation

Explain why model output is normalized and validated before it becomes a candidate.

Milestone 13 — Review Queue and Human Review Workspace

Relative size: XL

Objective

Deliver the main operational human-review experience.

User-visible outcome

Staff can:

- open Review Queue
- inspect the source PDF
- view evidence
- correct fields
- mark values Not Present
- resolve warnings
- add or remove supported list items

- see approval readiness

Included scope

- Review Queue
- filtering and sorting
- review workspace
- two-panel desktop layout
- source access
- field states
- evidence navigation
- correction endpoints
- warning resolution
- field decisions
- vehicle/driver/insured changes
- stale handling
- readiness
- responsive fallback
- accessibility

Excluded scope

- approval
- Reprocessing
- Correction Review
- full mobile approval

Dependencies

Milestone 12.

Database work

Review edits, decisions, resolutions, item changes.

Backend work

Granular Review API.

Worker work

None beyond completed processing.

Frontend work

Full Review Workspace.

Test requirements

Blocking:

- every review mutation
- stale version
- target ownership
- readiness recalculation
- undo behavior
- warning resolution
- PDF failure independent from field loading
- keyboard behavior
- focus management
- no full-review bulk update

Security checks

- evidence and source authorization
- no raw provider data
- no cross-document target mutation

Expected artifacts

- completed review vertical slice
- accessibility test notes
- two-user conflict demonstration

Definition of Done

A reviewer can transform an imperfect candidate into a server-persisted ready review without changing the candidate.

Human approval checkpoint

Complete a warning-heavy review manually and inspect the correction history.

Rollback or recovery

Individual edits can be superseded or undone. The baseline candidate remains untouched.

Interview explanation

Explain candidate versus effective review values and optimistic concurrency.

Milestone 14 — Approval and immutable policy versions

Relative size: L

Objective

Convert a Ready review into an immutable approved policy safely.

User-visible outcome

Staff can:

- inspect Approval Summary
- approve as Current or Historical
- optionally update customer contact address
- view Approved Summary
- view Current Policy and Policy History

Included scope

- server-derived approval summary
- readiness recheck
- customer freshness
- Current-policy freshness
- immutable policy copy
- approved children
- Current selection
- historical approval
- audit
- repeated approval
- rollback

Excluded scope

- email
- Correction Review
- reprocessing

Dependencies

Milestone 13.

Database work

Policy versions, children, Current selection.

Backend work

Approval transaction and policy APIs.

Worker work

None.

Frontend work

Approval drawer, approved summary, policy history.

Test requirements

Blocking:

- approval rollback
- repeated approval
- concurrent approval
- Current-policy race
- Historical approval
- contact-address update
- immutable records
- source relationships
- one Current invariant

Security checks

- final values derived server-side
- frontend cannot supply arbitrary approved fields
- audit is same transaction

Expected artifacts

- transaction test report
- policy-history vertical slice
- Current replacement preparation

Definition of Done

Approval is all-or-nothing and creates exactly one immutable version.

Human approval checkpoint

Inspect a successful approval and an intentionally failed approval transaction.

Rollback or recovery

Any error leaves prior Current policy and review state unchanged.

Interview explanation

Explain copy-on-approval and why Current is represented by a separate pointer.

Milestone 15 — Technical Retry, Reprocessing, Correction Review, and concurrency

Relative size: L

Objective

Complete the approved recovery and revision lifecycle.

User-visible outcome

Staff can:

- retry eligible technical failures
- intentionally reprocess
- preserve approved data during reprocessing
- create a Correction Review
- approve a corrected version
- handle stale reviews and Current-policy conflicts

Included scope

- Technical Retry route
- two-retry limit
- Reprocessing
- reusable artifacts
- active-attempt conflicts
- approved composite states
- Correction Review
- abandonment
- corrected policy relationships
- stale-data workflows

Excluded scope

- automatic provider fallback
- direct rejection reversal
- reassignment of source document to another customer

Dependencies

Milestones 11–14.

Database work

Attempt relationships, correction review, policy relationships.

Backend work

Eligibility and transaction rules.

Worker work

Artifact reuse and new attempts.

Frontend work

Recovery actions, warnings, comparison, stale states.

Test requirements

Blocking:

- third Technical Retry rejected
- active attempt conflict
- failed reprocessing preserves Current
- successful reprocessing creates new review
- Correction Review creates no attempt
- newer Current makes correction stale
- corrected approval creates new version

Security checks

- paid-processing confirmation
- backend usage check
- no bypass through direct endpoint call

Expected artifacts

- retry/reprocess lifecycle test
- Correction Review walkthrough
- composite-state verification

Definition of Done

Every approved recovery action has distinct state, cost, and history.

Human approval checkpoint

Run one Technical Retry, one Reprocessing, and one Correction Review.

Rollback or recovery

Failed new attempts never alter existing approved data.

Interview explanation

Explain the difference between Technical Retry, Reprocessing, replacement upload, and Correction Review.

Milestone 16 — Audit, usage controls, and acknowledgement email

Relative size: L

Objective

Complete business traceability, cost enforcement, and post-approval communication.

User-visible outcome

Staff can:

- inspect activity
- view usage
- see limits
- preview acknowledgement
- request send
- inspect delivery status
- deliberately resend

Included scope

- audit timelines
- filters
- usage summary
- reservations
- final usage
- live/Stored/simulated distinction
- processing-disabled state
- fixed email content
- background email job
- duplicate-send warning
- simulated demo email

Excluded scope

- arbitrary email editing
- billing
- invoices
- owner-facing limit editor
- notification center

Dependencies

Milestones 14–15.

Database work

Audit, usage, email.

Backend work

Activity, Usage, Email APIs.

Worker work

Email job and usage finalization.

Frontend work

Activity, Usage, preview, send status.

Test requirements

Blocking:

- same-transaction audit
- PII minimization
- usage reservation race
- limit reached
- Stored Result not live
- email idempotency
- email failure preserves approval
- resend relationship

Security checks

- recipient derived from customer
- no arbitrary subject/body
- email abuse limits
- provider message hidden

Expected artifacts

- audit-event catalog
- usage-limit test report
- email failure demonstration

Definition of Done

Important business activity is explainable, paid work is controlled, and email remains independent of approval.

Human approval checkpoint

Inspect audit records and usage from one complete workflow.

Rollback or recovery

Email can fail and retry without changing policy state. Reservations can be released safely.

Interview explanation

Explain audit versus logs and reservation versus finalized usage.

Milestone 17 — Stored Results and isolated demo sessions

Relative size: L

Objective

Create a reliable public portfolio demonstration.

User-visible outcome

A visitor can:

- enter an isolated demo session
- select a synthetic sample
- replay processing
- review and approve
- simulate email
- reset the demo
- remain unaffected by other visitors

Included scope

- template sets

- Stored Result templates
- demo session creation
- session ownership
- sample list
- replay job
- guided sample
- reset
- expiry
- cleanup
- simulated usage
- simulated email
- provider-outage-safe path

Excluded scope

- arbitrary public upload
- real email
- real customer data
- general multi-tenancy
- permanent demo user changes

Dependencies

Milestones 8, 12–16.

Database work

Demo sessions, templates, isolation constraints.

Backend work

Demo APIs and mandatory scoping.

Worker work

Replay, reset, expiry, cleanup.

Frontend work

Demo entry, guided path, labels, reset.

Test requirements

Blocking:

- cross-session denial
- reset idempotency
- template immutability

- compatibility failure
- provider outage
- session expiry
- simulated actions labelled correctly

Security checks

- synthetic only
- no staff data references
- no arbitrary local upload
- no real email destination
- session rate limits

Expected artifacts

- synthetic template package
- public golden path
- demo reset test
- provider-outage demonstration

Definition of Done

Two concurrent demo users can complete independent workflows and reset without affecting shared templates or staff data.

Human approval checkpoint

Use two separate browser sessions and inspect the resulting records.

Rollback or recovery

Disable new demo sessions while preserving staff environment if isolation fails.

Interview explanation

Explain shared immutable templates versus disposable per-session business records.

Milestone 18 — Security, observability, cleanup, and failure recovery

Relative size: L

Objective

Harden the application for public exposure.

User-visible outcome

Failures are clear, recoverable where appropriate, and accompanied by safe reference identifiers.

Included scope

- structured logs
- error tracking
- health checks
- worker heartbeat monitoring
- stale-job monitoring
- cleanup schedules
- raw-artifact expiry
- upload cleanup
- demo cleanup
- usage release
- rate limiting
- PII redaction
- signed-URL review
- dependency security review
- failure runbooks

Excluded scope

- formal compliance certification
- enterprise SIEM
- multi-region failover
- cryptographic audit ledger

Dependencies

All major workflows.

Database work

Cleanup indexes and diagnostic retention state.

Backend work

Error references, health, rate limits, redaction.

Worker work

Cleanup and maintenance jobs.

Frontend work

Failure Detail and safe guidance.

Test requirements

Blocking:

- storage outage
- provider outage
- database outage behavior
- web restart
- worker restart
- expired signed URL
- cleanup failure
- PII log scan
- rate-limit behavior

Security checks

Complete Release 1 security checklist.

Expected artifacts

- threat review
- incident runbook
- cleanup schedule
- monitoring dashboard
- failure matrix

Definition of Done

Critical failures are detectable, diagnosable, and do not silently corrupt business state.

Human approval checkpoint

Inspect logs for a failed attempt and verify no source text, policy number, address, VIN, prompt, or signed URL is present.

Rollback or recovery

Disable live processing or email independently while Stored Results and approved data remain usable.

Interview explanation

Explain how system outages affect web, worker, database, Storage, AI, OCR, and email differently.

Milestone 19 — End-to-end testing and deployment

Relative size: XL

Objective

Deploy and validate a production-like public-demo environment.

User-visible outcome

The complete synthetic golden path works on public infrastructure.

Included scope

- shared preview
- public demo
- migrations
- Vercel
- Render web
- Render worker
- Supabase demo project
- private Storage
- monitoring
- health checks
- end-to-end tests
- browser compatibility
- cold-start assessment
- deployment rollback
- release smoke tests

Excluded scope

- real-customer production
- real customer documents
- unapproved live provider allowance
- public marketing before readiness

Dependencies

Milestones 1–18.

Database work

Production-like demo migration and seed.

Backend work

Deployment configuration and smoke resources.

Worker work

Always-on or reliable public-demo worker behavior.

Frontend work

Production asset and API configuration.

Test requirements

Blocking:

- full Stored Result golden path
- approved limited live path when enabled
- authentication
- upload restrictions
- review
- approval
- email simulation
- reset
- worker recovery
- deployment smoke
- rollback
- accessibility critical path

Security checks

- production demo secrets
- public endpoint review
- no real data
- private Storage
- direct database access denied
- demo rate limits

Expected artifacts

- deployment checklist
- smoke-test results
- rollback procedure
- public-demo URL
- release candidate record

Definition of Done

The complete public-demo workflow passes from a clean session after deployment.

Human approval checkpoint

Perform the release checklist manually using a new browser profile.

Rollback or recovery

Return to last verified frontend, web, worker, and migration-compatible release.

Interview explanation

Explain deployment boundaries and why web and worker versions must remain compatible.

Milestone 20 — Loom-ready visual polish and public-demo release

Relative size: L

Objective

Make the approved product credible, clear, and demonstration-ready.

User-visible outcome

A polished public demo and final Loom-ready workflow.

Included scope

- visual consistency
- loading and error polish
- responsive review behavior
- accessibility corrections
- realistic synthetic data
- guided demo
- tooltip and help text
- evidence readability
- warning clarity
- approval summary polish
- usage clarity
- empty states
- sample reset
- performance improvements
- public disclaimer
- Loom script rehearsal

Excluded scope

- new product functionality
- unsupported documents
- speculative dashboard metrics
- Voice AI
- marketing claims unsupported by evaluation

Dependencies

Milestone 19.

Database work

No schema redesign.

Backend and worker work

Only confirmed defects or performance corrections.

Frontend work

Primary focus.

Test requirements

Blocking:

- complete browser golden path
- responsive desktop and narrow view
- keyboard use
- readable warnings
- no cropped PDF controls
- reset reliability
- public-demo cold-start tolerance
- final accuracy statements match benchmark

Security checks

- synthetic sample review
- no hidden real email or live upload path
- disclaimer accurate
- no secrets in frontend bundle

Expected artifacts

- release candidate
- final Loom script
- demonstration checklist
- known-limitations page or notes
- screenshots and sample-data inventory

Definition of Done

Release 1 passes all readiness gates and can be demonstrated without manual database repair, hidden setup, or misleading claims.

Human approval checkpoint

Complete three rehearsals:

1. Stored Result golden path
2. warning-heavy correction path
3. failure and recovery explanation

Rollback or recovery

Do not add late features. Revert polish changes that threaten stable workflow behavior.

Interview explanation

Deliver a complete architecture, trade-off, failure, and workflow explanation without relying on Codex output.

---

5. Vertical slices

Slice 1 — Sign in and create a customer

Flow

```text id="ev01hf" Sign in → Backend verifies identity → Membership resolved → Customer created → Audit event created → Customer Details opens

```
Real behavior required

- Auth
- database
- API
- idempotency
- duplicate warning
- frontend validation
- server validation
- audit

Value

Proves the application’s authorization and mutable business-data foundation.

Slice 2 — Upload a PDF and see Processing

```text id="11xfoj"
```

```
Select customer → Request signed upload → Private transfer
→ Confirm completion → Validate PDF → Create document
→ Create attempt → Reserve usage → Create job → Show Processing
```

Value

Proves direct Storage transfer and atomic workflow creation.

Slice 3 — Worker claims a job and records stages

```text id="l2xddx"
Available job → Worker claim → Lease → Running stage → Durable completion → Next stage → Browser polls status
```

```
**Value**

Proves asynchronous work survives browser closure and worker interruption.

## Slice 4 — Stored Result becomes Needs Review

```text id="nyumv8"
Select sample → Create Stored Result attempt
→ Replay durable stages → Publish candidate
→ Create warnings and evidence → Create review → Needs Review
```
```

Value

Proves the full candidate contract without provider dependency.

Slice 5 — Inspect evidence and correct a field

```text id="qih23r"
Open review → Load fields and PDF independently → Select warning → Jump to evidence page → Save correction → Recalculate readiness
```

```
**Value**

Proves candidate immutability and editable review state.

## Slice 6 — Resolve warnings and reach Ready

```text id="i3fii1"
Mark optional field Not Present
→ Correct important warning
```
```

```
→ Confirm permitted identity difference
→ Backend recalculates readiness → Ready
```

Value

Proves warning semantics and approval gates.

Slice 7 — Approve and create a Current policy

```text id="mrbicn"
Open approval summary → Confirm Current → Recheck freshness → Copy final values → Create immutable version → Select Current → Finalize review → Audit → Approved Summary
```

```
**Value**

Proves the central business transaction.

## Slice 8 — Replace Current while preserving history

```text id="d99bq9"
Review newer declaration → Compare with Current
→ Approve as Current → Replace pointer
→ Previous version remains Historical
```

Value

Proves versioned policy history.

Slice 9 — Send or simulate acknowledgement

```text id="grdfq4"
Preview fixed acknowledgement → Confirm → Create email delivery → Background job → Sent / Failed / Simulated → Audit
```

```
**Value**

Proves independent external delivery.

## Slice 10 — Reset isolated demo session

```text id="j1lbzs"
Complete demo workflow → Reset
→ Dispose session records → Re-seed from templates
→ Return to known Demo Overview
```

Value

Proves public-demo reliability and isolation.

---

6. Controlled Document AI evaluation spike

6.1 Objective

Select one production-ready Document AI configuration through evidence rather than intuition.

6.2 Minimal tooling required

The spike needs only:

- benchmark manifest reader
- private document loader
- ground-truth reader
- parser adapters
- OCR adapters
- AI model adapters
- context preparation
- canonical candidate validator
- evidence verifier
- metric calculator
- latency and usage recorder
- repeatability runner
- result exporter
- configuration identity
- controlled raw-result retention

It does not need:

- public UI
- product database benchmark tables
- runtime routing
- fallback orchestration
- generalized experiment dashboard
- production retry infrastructure
- automatic prompt optimization

6.3 Step 1 — Benchmark preparation

Prepare:

- development set
- tuning set
- provider-comparison set
- hidden holdout
- failure corpus
- public synthetic demo samples

Every document must have:

- stable benchmark ID
- source rights classification
- document category
- scan category
- layout notes
- expected difficulty
- allowed provider transmission status

6.4 Step 2 — Ground truth

Manually annotate:

- supported field values
- source representation
- page
- excerpt
- field state
- vehicles
- drivers
- named insureds
- coverage relationships
- ambiguity
- missing values
- unsupported classification
- multiple-policy classification

Critical values receive a second manual review.

6.5 Step 3 — Parser comparison

Compare approved parser candidates using identical documents.

Measure:

- successful open rate
- page count
- direct text
- reading order
- word/block positions
- hidden duplicate text
- table preservation
- resource use
- rendering
- evidence support

Reject a parser when it cannot reliably support page-aware evidence or deployment constraints.

6.6 Step 4 — OCR comparison

Use the same selected scan pages and identical ground truth.

Measure:

- policy number
- VIN
- dates
- coverage amounts
- table-row preservation
- orientation
- evidence coordinates
- technical failures
- latency
- resource use
- cost
- privacy implications

Do not tune the AI extraction differently for each OCR provider during the first comparison.

6.7 Step 5 — Model comparison

Freeze:

- candidate contract
- extraction instruction version
- input context
- OCR artifacts
- normalization
- validation
- evidence rules

Compare shortlisted models on identical inputs.

Measure:

- field accuracy
- missing behavior
- list completeness
- associations
- structure
- evidence
- warning triggers
- repeatability
- latency
- usage
- cost
- refusals
- timeouts

6.8 Step 6 — Text-only versus hybrid input

Compare:

- page-aware text only
- page-aware text plus selected images
- native full-PDF input on a limited subset where justified

The hybrid approach wins only when it materially improves:

- table relationships
- vehicle associations
- deductible associations
- evidence
- critical-field quality

without unacceptable cost, privacy, latency, or repeatability penalties.

6.9 Step 7 — Evidence testing

Personally inspect:

- page correctness
- excerpt support
- geometry correctness
- relationship context
- model-proposed evidence verification
- fabricated excerpt attempts
- page-only fallbacks

Any fabricated evidence presented as verified blocks the configuration.

6.10 Step 8 — Repeatability

Run selected difficult documents repeatedly.

Blocking variations include:

- policy number
- date
- named insured
- vehicle list
- coverage limit
- deductible association
- warning severity reduction
- evidence page error

6.11 Step 9 — Latency and cost

Record:

- parser time
- OCR time
- model time
- repair time
- total time
- OCR pages
- input usage
- image usage
- output usage
- failed-call usage
- average and slower-tail cost

Do not finalize cost ceilings until measured data exists.

6.12 Step 10 — Hidden holdout

The hidden set is evaluated after the provisional winner is selected.

No tuning may occur before its first result is recorded.

6.13 Evaluation outcomes

Pass

A configuration satisfies all mandatory release gates.

Action:

- approve configuration
- create approved pipeline version
- productionize only that path

Revise

The pipeline is close but a bounded issue remains.

Examples:

- warning recall
- one OCR preprocessing problem
- evidence matching
- context selection

Action:

- make one controlled change
- rerun affected regression and holdout rules

Reject

Examples:

- silent critical errors
- unstable policy identity
- wrong vehicle associations
- fabricated evidence
- excessive failures
- unacceptable privacy
- unacceptable cost

Action:

- reject configuration
- evaluate the next candidate
- do not hide failure through UI warnings alone

6.14 Evidence the owner must inspect

The owner must inspect:

- raw source page
- ground truth
- candidate output
- normalized value

- evidence
- warning
- reviewer consequence
- configuration identity

for every critical failure.

6.15 Decision-log update

The provider-selection record must contain:

- selected parser
- OCR path
- model
- input strategy
- instruction version
- quality results
- holdout results
- repeatability
- latency
- cost
- privacy
- retention
- rejected alternatives
- known limitations
- owner approval date

6.16 Approved pipeline version

The winning configuration becomes one immutable pipeline-version record.

Future changes require:

- new version
- regression evidence
- owner approval
- no mutation of historical attempts or approved policies

---

7. Migration sequence

7.1 Recommended order

1. Migration metadata and extensions
2. Agencies

3. Application users
4. Agency memberships
5. Demo-session foundation required for ownership references
6. Customers
7. File objects
8. Upload authorizations
9. Pipeline versions
10. Documents
11. Processing attempts
12. Background jobs
13. Background-job executions
14. Processing-stage executions
15. Processing artifacts
16. Stage-artifact links
17. Document pages
18. Extraction candidates
19. Candidate policy fields
20. Candidate named insureds
21. Candidate vehicles
22. Candidate drivers
23. Candidate coverages
24. Candidate alternatives
25. Candidate evidence
26. Alternative-evidence links
27. Validation signals
28. Generated warnings
29. Warning-signal links
30. Reviews
31. Review corrections
32. Field decisions
33. Warning resolutions
34. Review item changes
35. Approved policy versions
36. Approved policy child tables
37. Current-policy selection
38. Audit events
39. Usage limits
40. Usage reservations
41. Usage records
42. Email deliveries
43. Demo template sets
44. Stored Result templates
45. Idempotency records
46. Operational indexes
47. Partial unique constraints
48. final ownership and consistency constraints

7.2 Constraints added after supporting behavior

The following may initially be introduced after the relevant tables and transactions exist:

Active-attempt partial uniqueness

Add before upload completion is enabled.

Current-policy uniqueness

Add before approval is enabled.

One primary named insured

Add before candidate publication.

Review baseline exclusivity

Add before review creation.

Stored Result mode consistency

Add before demo replay.

Completed-attempt terminal checks

Add before live worker processing.

Strict non-null approved fields

Approved policy tables may begin nullable during migration development, but must become required before approval is enabled.

7.3 Migration principles

- Additive first
- no destructive renaming through column reuse
- nullable before required
- backfill before constraint
- test empty and populated upgrades
- web and worker declare compatible migration range
- migration rollback is not assumed
- application rollback must tolerate additive schema

---

8. Testing roadmap

8.1 Testing layers

Test layer	Begins	Blocks
Schema and constraint tests	Milestone 3	Every persistence milestone
Migration tests	Milestone 3	Shared environment deployment
API contract tests	First API milestone	Each API milestone
Authorization tests	Milestone 4	Every protected feature
Transaction rollback tests	Upload milestone	Upload, approval, retry, demo reset
Idempotency tests	Customer creation	Every repeat-sensitive action
Concurrency tests	Customer update	Review, approval, Current replacement
Worker claim tests	Milestone 7	Any background processing
Lease/reclaim tests	Milestone 7	Public processing
Stage fixture tests	Milestone 7	Pipeline implementation
Candidate fixture tests	Milestone 8	Review development
Document AI benchmark tests	Milestone 9	Provider selection
Pipeline regression tests	Milestone 11	Live pipeline release
Readiness tests	Milestone 13	Approval
Approval transaction tests	Milestone 14	Policy workflow release
Policy-version tests	Milestone 14	Current/history UI
Retry/Reprocessing tests	Milestone 15	Recovery release
Demo-isolation tests	Milestone 17	Public demo
Signed Storage tests	Milestone 6	Upload and PDF viewer
End-to-end browser tests	Milestone 5 onward	Public release
Accessibility tests	Milestone 5 onward	Milestones 13 and 20
Deployment smoke tests	Milestone 19	Public release
Security regression tests	Milestone 18	Public release

8.2 Blocking versus deferred tests

Must block the relevant milestone

- foreign-key and uniqueness tests
- authorization
- transaction rollback
- idempotency
- concurrency
- worker claims
- lease reclaim
- candidate immutability
- approval rollback
- Current-policy uniqueness
- demo isolation
- signed Storage authorization
- silent critical benchmark errors
- Stored Result compatibility

May mature later but must exist before public release

- broad visual regression
- performance baseline
- slower browser matrix
- noncritical accessibility refinements
- extensive search-volume tests
- long-running cleanup soak tests

8.3 Test-data policy

- Synthetic by default
- fixed IDs only where tests require deterministic references
- no copied client documents
- no real customer email
- provider fixtures sanitized
- hidden holdout inaccessible to prompt-tuning tasks

8.4 Test ownership

Codex may write tests, but the owner must verify:

- the test asserts the real invariant
- the test would fail under the known bad behavior
- mocks do not bypass the transaction or authorization being tested
- end-to-end tests use real APIs and database where intended

---

9. Codex governance rules

Every Codex task must follow these rules.

9.1 One bounded objective

Each task must have one primary outcome.

Bad task:

> Build customers, uploads, processing, and review.

Acceptable task:

> Implement idempotent customer creation against the approved customer contract.

9.2 Read approved specifications first

The task must name the controlling sections.

Codex must not infer behavior from similar SaaS products.

9.3 Inspect before editing

Codex must inspect:

- relevant existing implementation
- migrations
- contracts
- tests
- naming
- current decision log

before proposing changes.

9.4 Propose a short plan

Before editing, Codex should report:

- files or areas expected to change
- approach
- assumptions
- tests
- risks

- excluded work

9.5 Identify contradictions

Codex must stop before implementation when:

- repository behavior contradicts approved documents
- two approved documents conflict
- a migration makes an approved invariant impossible
- an API contract lacks necessary persistence

It should describe the smallest correction, not silently choose.

9.6 No silent scope or architecture changes

Codex must not:

- add a new service
- add another database
- introduce a queue provider
- change an endpoint semantic
- merge candidate and approved data
- replace granular review writes with one bulk update
- add unsupported fields
- add dynamic roles
- add multi-tenancy

without explicit owner approval.

9.7 Package discipline

Every new package must include:

- purpose
- why existing dependencies are insufficient
- maintenance risk
- security implications
- lighter alternative

9.8 No unrelated refactors

Do not combine feature work with:

- project-wide renaming
- formatting unrelated files
- dependency upgrades
- abstraction rewrites

- folder movement
- performance refactors

9.9 Reviewable diffs

A task should produce a diff small enough for the owner to understand.

When it becomes too large:

- stop
- preserve current state
- propose a split

9.10 Migrations and tests travel with behavior

A persistence change must include:

- migration
- constraint tests
- contract changes
- rollback/compatibility note

9.11 Run relevant checks

Codex should run:

- formatter
- lint
- type checks
- targeted tests
- migration checks
- broader tests when shared contracts changed

9.12 Completion report

Every task reports:

- changed files
- implemented behavior
- tests run
- results
- assumptions
- unresolved issues
- migrations
- operational concerns
- manual verification steps

9.13 Honesty

Codex must state when:

- a test fails
- an integration was not run
- provider credentials were unavailable
- an assumption remains
- a migration was not applied
- behavior is fixture-only

9.14 Stop at the boundary

Codex must not implement the next roadmap item opportunistically.

9.15 Data and secret safety

Codex must never:

- commit secrets
- commit real policy documents
- commit personal customer data
- place raw provider responses in logs
- paste credentials into tests
- use production data for local development
- make public Storage objects

---

10. Human-supervised Codex workflow

10.1 Repeating loop

Step 1 — Select one roadmap work package

Choose the smallest task that creates one testable result.

Step 2 — Prepare a bounded Codex prompt

Include:

- task objective
- controlling source sections
- included and excluded scope
- acceptance criteria
- required tests

- prohibited decisions

Step 3 — Let Codex inspect the repository

Codex should read before proposing edits.

Step 4 — Review its proposed plan

Check:

- scope
- affected areas
- assumptions
- packages
- migration impact
- security
- tests

Step 5 — Approve or correct the plan

Do not allow implementation while the plan contains hidden redesign.

Step 6 — Let Codex implement

Codex performs only the approved plan.

Step 7 — Run tests and checks

Run targeted checks first, then shared checks when contracts changed.

Step 8 — Review the diff manually

Inspect:

- migration
- transaction
- authorization
- errors
- tests
- naming
- data exposure
- unnecessary abstractions

Step 9 — Ask Codex to explain the implementation

The explanation should cover:

- data flow

- invariant protection
- failure behavior
- concurrency
- idempotency
- trade-offs

Step 10 — Verify behavior in the product

Use the real interface or API.

Do not rely only on passing unit tests.

Step 11 — Update decision and implementation notes

Record only meaningful changes.

Step 12 — Commit after approval

One intentional commit for the approved work package.

10.2 Reject the work when

- it changes scope silently
- tests only confirm mocks
- authorization is missing
- frontend becomes business authority
- migration is destructive without approval
- raw provider output enters business tables
- candidate data mutates
- approved data mutates
- an unapproved dependency is introduced
- error behavior violates the specification
- the diff cannot be understood

10.3 Revert when

- a merged task violates an invariant
- a migration cannot be safely corrected forward at the current project stage
- the feature creates data corruption
- security boundaries are breached
- worker duplication creates several outcomes

10.4 Split when

- more than one aggregate changes
- several unrelated endpoints are added
- frontend, backend, worker, and migration changes cannot be reviewed as one coherent behavior

- a provider integration and product workflow are mixed
- broad refactoring appears
- the task requires a new architectural decision

---

11. Codex work-package template

Task identity

Task ID:
Title:
Milestone:
Vertical slice:

Objective

State one observable outcome.

Source-document references

List exact approved documents and relevant sections.

Current repository state

Describe:

- existing capability
- known relevant files or areas
- current tests
- current migrations
- unresolved defects

Included work

List only required behavior.

Excluded work

List adjacent work Codex must not perform.

Acceptance criteria

Define measurable outcomes.

Required tests

Specify:

- unit
- contract
- transaction
- authorization
- concurrency
- idempotency
- browser
- manual

as relevant.

Expected areas to change

Identify likely areas without requiring Codex to change all of them.

Prohibited decisions

Examples:

- no new package without approval
- no endpoint redesign
- no provider selection
- no schema broadening
- no bulk review update
- no new service

Human-review checklist

The owner should inspect:

- plan
- migration
- diff
- tests
- data flow
- authorization
- failure behavior
- logs
- interface behavior

Completion report requirements

Codex must report:

- changed areas
- decisions
- tests
- failures
- remaining issues
- manual verification
- migration and deployment notes

Rollback notes

State:

- how to disable
- how to revert before shared data
- forward-fix expectations after shared migration
- data cleanup required

---

12. Review and approval gates

Gate 1 — Repository structure

Inspect:

- one repository
- separate runtime responsibilities
- dependency count
- commands
- CI
- secret rules

Approval evidence:

A clean checkout runs.

Gate 2 — Migration foundation

Inspect:

- schema catalog match
- migration order

- constraints
- JSONB use
- empty-database application
- populated forward migration

Approval evidence:

Constraint tests pass.

Gate 3 — Authentication

Inspect:

- token verification
- membership
- actor identity
- 401/403/404
- demo context

Approval evidence:

Cross-context access fails.

Gate 4 — Signed Storage access

Inspect:

- generated object path
- expiry
- private bucket
- completion verification
- signed view
- arbitrary-path rejection

Approval evidence:

A valid upload works and public access fails.

Gate 5 — Worker claims and recovery

Inspect:

- claim exclusivity
- lease
- heartbeat
- restart
- duplicate stage

- dead-letter behavior

Approval evidence:

A killed worker recovers without duplicate outcome.

Gate 6 — Evaluation corpus

Inspect:

- document diversity
- rights
- ground truth
- hidden holdout
- critical second review

Approval evidence:

Benchmark manifest is frozen.

Gate 7 — Provider selection

Inspect:

- silent errors
- associations
- evidence
- repeatability
- latency
- cost
- privacy

Approval evidence:

Signed Go decision and approved pipeline version.

Gate 8 — Candidate publication

Inspect:

- immutable candidate
- supported fields only
- evidence ownership
- warnings
- Stored/live parity

Approval evidence:

Candidate publication transaction passes rollback tests.

Gate 9 — Review state

Inspect:

- corrections
- Not Present
- warning resolution
- list changes
- readiness
- stale conflict

Approval evidence:

Two-browser conflict behaves correctly.

Gate 10 — Approval transaction

Inspect:

- server-derived final values
- rollback
- repeated approval
- audit
- immutable rows

Approval evidence:

All-or-nothing transaction tests.

Gate 11 — Current-policy replacement

Inspect:

- previous Current
- replacement pointer
- history
- concurrency

Approval evidence:

Only one Current exists after racing requests.

Gate 12 — Demo isolation

Inspect:

- two sessions
- template immutability
- reset
- expiry
- no staff references

Approval evidence:

Cross-session database and API tests pass.

Gate 13 — Security

Inspect:

- authorization
- private files
- secrets
- logs
- raw artifacts
- rate limits
- cleanup

Approval evidence:

Security checklist and PII log scan pass.

Gate 14 — Deployment

Inspect:

- compatible frontend/web/worker versions
- migrations
- health
- rollback
- public demo secrets

Approval evidence:

Release smoke test passes after clean deployment.

Gate 15 — Public launch

Inspect:

- release-readiness checklist
- benchmark claims
- demo reset
- sample quality
- accessibility
- Loom rehearsal

Approval evidence:

Owner signs Release 1 ready.

---

13. Documentation and learning during implementation

Maintain only lightweight records.

13.1 Implementation decision log

Record:

- decision
- context
- selected option
- alternatives
- trade-off
- owner
- date
- affected source document
- revisit condition

13.2 Architecture deviations

Only record when implementation differs from an approved document.

A deviation must include:

- reason
- smallest correction
- owner approval

- migration or compatibility impact

13.3 Evaluation results

Keep:

- configuration
- benchmark version
- metrics
- errors
- decision
- raw artifact references

outside the production database.

13.4 Milestone completion notes

One short record per milestone:

- delivered behavior
- tests
- known limitations
- unresolved risks
- demonstration steps
- next dependency

13.5 Known limitations

Maintain a concise list such as:

- supported document type
- page and size limits
- readable scans only
- one policy per file
- no carrier verification
- no coverage advice
- public demo uses synthetic data
- Stored Result labels

13.6 Incident and failure notes

Record only meaningful failures:

- what happened
- impact
- detection

- recovery
- root cause
- prevention

13.7 Interview explanation notes

After each gate, write:

- what was built
- why this design was chosen
- simpler alternative
- accepted trade-off
- failure behavior
- future scaling path

13.8 Loom script notes

Maintain:

- opening business problem
- golden path
- one human correction
- one warning explanation
- approval
- policy history
- audit
- usage
- architecture summary
- limitations

---

14. Environment and deployment roadmap

14.1 Local development

Configured in Milestone 2.

Uses:

- synthetic data
- local or isolated PostgreSQL
- local Auth-compatible behavior
- private development Storage
- Stored Results
- fixture provider responses

- simulated email

Live provider calls require explicit enablement.

14.2 Automated testing

Configured beginning in Milestones 2–3.

Requires:

- disposable database
- isolated Storage test location
- no live AI or OCR
- deterministic fixtures
- migration from empty database
- repeatable cleanup

14.3 Shared preview

Introduce after the customer and upload slices are stable.

Purpose:

- integrated frontend/backend review
- owner testing
- migration validation
- no public users

Use:

- synthetic data
- Stored Results
- restricted access
- separate secrets

14.4 Public demo

Configure during Milestones 17–19.

Services:

- Vercel frontend
- Render web service
- Render worker
- Supabase demo project
- private Storage
- monitoring

- error tracking
- cleanup schedules
- simulated email
- Stored Results default
- optional limited live sample after provider approval

14.5 Future real-customer production

Not part of the public-demo release.

Before use, it requires:

- dedicated production Supabase project
- dedicated secrets
- production retention policy
- malware scanning
- customer-data deletion procedure
- provider terms review
- backup and recovery review
- support and incident process
- no demo-reset capability
- real email configuration

14.6 Service configuration timing

Service	Configure
Supabase local/development	Milestone 2
Supabase shared preview	After Milestone 6
Supabase public demo	Milestone 17–19
Vercel preview	Milestone 5 or 6
Vercel public demo	Milestone 19
Render web preview	Milestone 6–7
Render worker preview	Milestone 7
Render public demo	Milestone 19
Private Storage	Milestone 2 and finalized in 6
Resend development/simulation	Milestone 16
Resend real production	Future customer environment
Error tracking	Basic in Milestone 1, full in 18

Service	Configure
Monitoring	Milestone 18
Health checks	Milestones 1, 7, and 19
Cleanup schedules	Milestones 17–18
Production-like secrets	Milestone 19

14.7 Migration deployment

- migrations apply before code depends on new schema
- web and worker compatibility is checked
- destructive migration is prohibited without owner approval
- Stored Result templates are validated after migration
- smoke tests run after migration and before public promotion

---

15. Release-readiness checklist

15.1 Functional completeness

- Customer workflow works.
- Upload works.
- Processing survives browser closure.
- Needs Review is created.
- Review corrections persist.
- Warnings resolve correctly.
- Approval creates immutable policy.
- Current and Historical behavior works.
- Technical Retry works.
- Reprocessing works.
- Correction Review works.
- Audit works.
- Usage works.
- email simulation works.
- Demo reset works.

15.2 Document AI quality

- Approved benchmark version recorded.
- Clean-PDF gate passes.
- scanned-PDF gate passes.
- unsupported-document gate passes.
- multiple-policy gate passes.
- vehicle associations pass.

- evidence gates pass.
- repeatability gates pass.

15.3 Silent critical errors

- Zero silent critical errors on release-gate corpus.
- Zero silent critical errors on hidden holdout.
- Any critical variation is blocked or warned appropriately.

15.4 Security

- Browser has no service credentials.
- Business tables are not directly browser-accessible.
- PDFs are private.
- signed URLs expire.
- object authorization is tested.
- public demo cannot upload arbitrary files.
- rate limits are active.
- no real data exists in demo.

15.5 Privacy

- Logs contain no source text, address, policy number, VIN, prompt, raw response, or signed URL.
- Raw artifacts are restricted.
- Artifact expiry is active.
- Provider transmission is documented.
- Public claims avoid formal compliance language.

15.6 Reliability

- Worker interruption recovers.
- web restart does not lose jobs.
- provider failure creates safe Failed state.
- email failure preserves approval.
- Storage failure blocks source-dependent actions.
- Stored Result path survives provider outage.

15.7 Idempotency

Repeated:

- customer creation
- upload completion
- approval
- rejection
- retry
- Reprocessing

- email
- reset

produces one business outcome.

15.8 Concurrency

- stale customer update rejected
- stale review edit rejected
- concurrent approval safe
- Current-policy race safe
- stale Correction Review safe

15.9 Worker recovery

- expired lease reclaims
- completed stages are reused
- duplicate execution does not duplicate candidate
- poisoned job becomes visible
- stale attempt is detected

15.10 Demo reset

- known seed state restored
- other sessions unaffected
- templates unchanged
- repeated reset safe
- expired session cleaned

15.11 Cost controls

- usage reserved before live work
- configured limits enforced
- approaching-limit state works
- limit reached blocks new live processing
- Stored Results remain available
- simulated usage is labelled

15.12 Observability

- request IDs
- correlation IDs
- attempt trace
- stage duration
- provider failure category
- worker heartbeat
- stale-job alert

- error reference
- health checks

15.13 Accessibility

- keyboard navigation
- visible focus
- labelled fields
- warning severity not color-only
- status announcements
- PDF controls accessible
- narrow-screen fallback clear

15.14 Responsive behavior

- desktop review works
- narrow review uses approved stacked/tab layout
- no hidden blocking actions
- mobile does not pretend unsupported full review is available

15.15 Deployment

- public demo environment separate
- migrations complete
- web/worker compatible
- rollback tested
- health checks pass
- cleanup schedules active

15.16 Sample data

- synthetic only
- several realistic layouts
- clean, scanned, warning, failed, approved, and history samples
- no trademark or personal-data issues
- Stored Result labels accurate

15.17 Loom readiness

- public URL stable
- no avoidable cold start
- guided sample resets
- script rehearsed
- failure backup sample available
- benchmark claims documented
- limitations ready to explain
- screen recording lighting, resolution, audio, and crop tested

16. Risks and scope control

Risk	Mitigation	Warning sign
Too much infrastructure first	Require visible vertical slices	Several weeks pass without a user workflow
UI built before contracts stabilize	Implement against approved API and fixture contracts	Frontend stores business truth locally
Evaluation spike delayed	Make provider gate part of critical path	Live provider code appears before benchmark
Benchmark overfitting	Hidden holdout and frozen configurations	Instructions mention specific sample layouts
Codex makes hidden decisions	Mandatory plan and decision list	New package/schema/status appears without approval
Oversized diffs	One objective and diff-size review	Task changes many aggregates
Weak tests	Tests required in work package	“Works manually” is the only evidence
Frontend/backend divergence	Contract tests and generated examples	Different field names or status meanings
Web/worker mismatch	Payload and handler versions	Worker cannot read newly created jobs
Demo/staff leakage	Session ownership tests	Demo query lacks session filter
Uncontrolled API cost	Usage reservation and disabled-by-default live calls	Provider calls happen during normal UI development
Premature polishing	Gate polish after workflow stability	Visual refactor delays approval transaction
Analysis paralysis	Time-box decisions with evidence gates	Rewriting plans without implementing approved foundation
Premature abstraction	Require two proven uses before generalizing	Generic framework appears for one document type
Raw artifact exposure	Restricted APIs and Storage	Raw response appears in review or logs
Migration debt	Review each migration before shared use	Compensating migrations appear before first feature

Risk	Mitigation	Warning sign
Too many dependencies	Package-justification rule	Several libraries solve overlapping needs
Fixture/live divergence	Same candidate publication contract	Separate mock-only review schema
False accuracy claims	Public claims tied to benchmark version	Marketing text says “accurate” without numbers
Public-demo instability	Stored Results and reset	Demo depends entirely on live AI
Owner stops understanding code	Explanation checkpoint after each task	Owner cannot describe transaction or data flow

17. Immediate first coding step

Recommendation

The first implementation milestone should be:

> **Create the runnable repository foundation with engineering guardrails and three minimal runtime identities: frontend, FastAPI web service, and worker.**

What should be created first

- one Git repository
- minimal Next.js runtime
- minimal FastAPI runtime
- minimal worker runtime
- environment validation
- health responses
- formatting
- linting
- type checking
- testing commands
- CI
- secret-exclusion rules
- project decision references
- Codex contribution rules

Why it comes first

Every later work package depends on:

- reliable commands
- version coordination
- quality checks
- environment handling
- reviewable diffs
- consistent frontend/backend/worker boundaries

It is small enough to inspect fully and useful enough to prevent future inconsistency.

What must not be included

- Supabase business integration
- customer tables
- authentication workflow
- upload
- queue logic
- AI packages
- OCR packages
- repository-wide abstractions
- UI design system
- deployment production configuration
- provider selection

Success looks like

- clean checkout
- documented local startup
- frontend starts
- backend starts
- worker starts
- health checks succeed
- automated checks pass
- missing configuration fails clearly
- no secrets are committed
- owner understands every dependency

What the owner must understand before proceeding

- why one repository was chosen
- which runtime owns which responsibility
- why the worker is separate from the web process
- how environment variables are divided
- how CI protects later Codex changes

- how to run every check personally
- how to inspect the next Codex diff

---

18. Final roadmap summary

18.1 Phase summary

Foundation

Phases 0–4 establish:

- decisions
- repository
- environments
- schema
- identity

First product slices

Phases 5–7 establish:

- customer workflow
- private upload
- durable worker processing

Provider-neutral contract and evaluation

Phases 8–10 establish:

- candidate fixtures
- benchmark corpus
- evaluation evidence
- provider-selection gate

Core live workflow

Phases 11–15 establish:

- approved live pipeline
- candidates
- review
- approval
- policy history
- retries
- Correction Review

Operational completeness

Phases 16–18 establish:

- audit
- usage
- email
- demo isolation
- security
- observability
- cleanup

Release

Phases 19–20 establish:

- deployment
- end-to-end validation
- polish
- public demo
- Loom readiness

18.2 Critical path

```
text id="g5pc0u"
Repository
→ Local environment
→ Core schema
→ Authentication
→ Customer
→ Upload
→ Job worker
→ Candidate fixture
→ Evaluation
→ Provider approval
→ Live pipeline
→ Review
→ Approval
→ Demo isolation
→ Security and deployment
→ Public release
```

18.3 Provider-selection gate

No final live Document AI integration begins until:

- benchmark corpus is approved
- ground truth is approved
- provider comparison is complete
- holdout is complete
- silent critical error rule passes
- cost is measured
- privacy is reviewed
- owner approves one pipeline version

18.4 Estimated major effort distribution

Relative effort across the complete build:

Area	Approximate share
Foundation, environments, migrations, Auth	15%
Customers, uploads, documents, worker	17%
Evaluation spike and provider selection	15%
Live Document AI pipeline	15%
Review Workspace and readiness	15%
Approval, policy versions, recovery	9%
Audit, usage, email, demo	7%
Security, observability, deployment, polish	7%

These are planning proportions, not delivery promises. Document complexity and evaluation results may shift them.

18.5 Major owner checkpoints

The owner must explicitly approve:

1. Repository foundation
2. Migration foundation
3. Authentication boundary
4. signed Storage flow
5. worker claim and recovery
6. benchmark corpus
7. provider selection

8. candidate publication
9. review state
10. approval transaction
11. Current-policy replacement
12. demo isolation
13. security readiness
14. deployment candidate
15. public release

18.6 Definition of Done for Release 1

Release 1 is done when:

- the approved golden path works end to end
- supported PDFs create reviewable candidates
- candidates never become approved automatically
- human review is evidence-backed
- approval is transactionally safe
- policy versions are immutable
- one Current policy is guaranteed
- failures preserve existing approved data
- retries and Reprocessing are distinct
- Correction Review creates no AI attempt
- audit history explains business actions
- cost limits are backend-enforced
- email is fixed and independent
- Stored Results support a provider-outage-safe demo
- demo sessions are isolated and disposable
- no real personal data exists in the public demo
- no silent critical benchmark errors exist
- worker recovery passes
- idempotency and concurrency pass
- private Storage is enforced
- logs and errors protect PII
- deployment and rollback are verified
- accessibility and responsive gates pass
- the owner can explain the architecture and trade-offs without relying on Codex

18.7 Exact Loom recording point

Internal learning recordings may begin after any successful vertical slice.

The **public portfolio Loom** should begin only after:

1. Milestone 20 is complete.
2. The public-demo release gate is signed.
3. A new isolated demo session completes the Stored Result golden path.

4. Reset is verified immediately afterward.
5. The public URL has passed a fresh deployment smoke test.
6. No visible debug data or test-only controls remain.
7. The benchmark claims and limitations have been reviewed.
8. One backup sample is ready in case the primary demonstration path fails.

At that point, the product is not merely visually impressive. It is a system the owner can confidently demonstrate, defend, and explain in proposals and interviews.